

CENG 463 Take-Home Midterm Report

Student Name: Abdalbaki Çeltik

Student ID: 290201072

Repository: github.com/bakiceltik/ceng463-take-home-midterm-Abdalbaki-celtik

April 2026

Contents

1	Introduction	2
1.1	Reproducibility Checklist	2
2	Question 1: Regression	2
2.1	Problem Setup	2
2.2	Models and Feature Engineering	2
2.3	Evaluation	3
2.4	Discussion	4
3	Question 2: Classification Under Extreme Imbalance	5
3.1	Problem Setup	5
3.2	Models and Sampling Strategies	5
3.3	Evaluation and Calibration	5
3.4	Discussion	6
4	Question 3: Dimensionality Reduction	7
4.1	Problem Setup	7
4.2	Methods	7
4.3	Evaluation	7
4.4	Discussion	9
5	Question 4: Clustering	9
5.1	Problem Setup	9
5.2	Methods	9
5.3	Evaluation	9
5.4	Discussion	11
6	Question 5: Neural Networks	12
6.1	Problem Setup	12
6.2	Models and Regularisation	12
6.3	Evaluation	12
6.4	Interpretability and Discussion	14
7	Conclusion	15

1 Introduction

This report addresses all five questions from the CENG 463 take-home midterm. The project is implemented with an emphasis on reproducibility, consistent random seeds, explicit preprocessing, and clear separation between exploratory work and report-ready experiments. The classical machine-learning pipelines use scikit-learn [16], and the neural-network experiments use PyTorch [15].

1.1 Reproducibility Checklist

- All experiments use explicit random seeds and stored configuration files.
- Model comparison uses fixed data splits unless a controlled comparison requires otherwise.
- Figures and tables are generated from code and stored in the repository outputs.
- Final test-set reporting is reserved for the last evaluation step.

2 Question 1: Regression

2.1 Problem Setup

For Question 1, I used the California Housing dataset from `sklearn`, originally described by Pace and Barry [14]. The dataset contains 20,640 observations, 8 numeric predictors, and a continuous target `MedHouseVal`. I kept a single holdout test split of 20% for the final evaluation and performed all model comparison on the remaining 16,512 training examples using repeated cross-validation, in line with the instruction that the test set should only be used once at the end.

Exploratory analysis showed that the target distribution was positively skewed, with training-set skewness equal to 0.9776, so I enabled a $\log(1+y)$ transform for the main non-robust models. The strongest linear relationship with the target was median income ($r = 0.6881$), followed by weaker associations for average rooms ($r = 0.1519$), latitude ($r = -0.1442$), and house age ($r = 0.1056$). Outlier analysis using both the IQR rule and Z-scores suggested that `AveBedrms`, `Population`, and `MedHouseVal` were the most problematic variables under the IQR criterion, with outlier rates of 6.83%, 5.83%, and 5.18%, respectively.

Figure 1 summarizes the main exploratory findings. The corresponding plots and tables were generated automatically by the experiment pipeline and stored under the Question 1 output directory.

2.2 Models and Feature Engineering

I compared six regression approaches:

1. Linear Regression,
2. RidgeCV,
3. LassoCV,
4. Elastic Net CV,
5. XGBoost Regressor with randomized hyperparameter search,
6. Huber Regressor as the robust baseline.

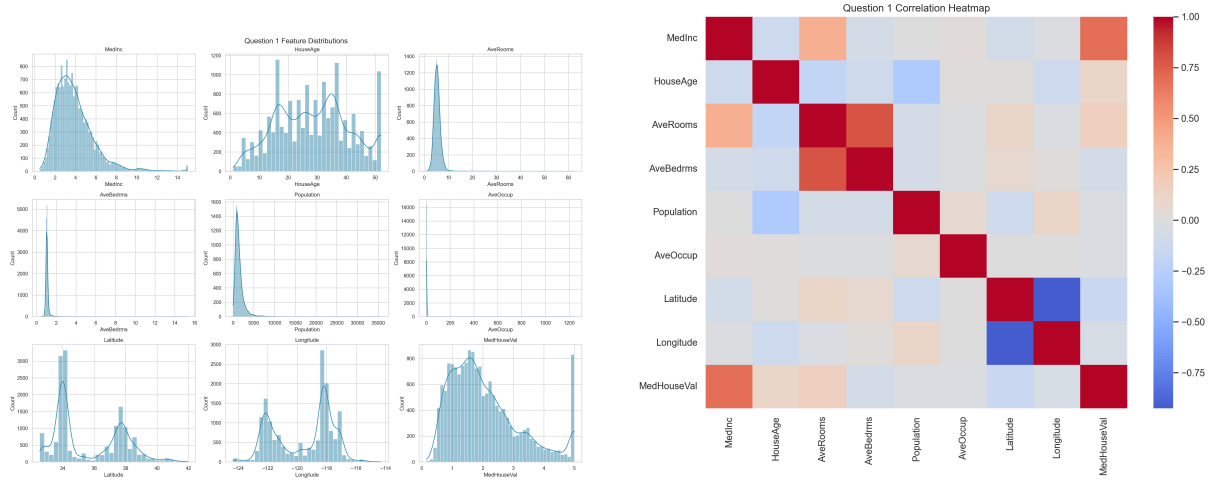


Figure 1: Exploratory analysis for the California Housing regression task. The left panel shows the feature and target distributions, while the right panel shows the full correlation heatmap.

Feature engineering was handled explicitly rather than implicitly. I generated polynomial and interaction features up to degree 3 to study the growth of the feature space and to support model selection. The raw 8 predictors expanded to 44 features at degree 2 and 164 features at degree 3. In the final modeling setup, degree-2 features were used for the linear and regularized models because they provided a more stable trade-off between expressiveness and numerical behavior than degree 3.

Recursive feature elimination was run on the polynomial feature space, and the selected set emphasized interaction-heavy terms involving *AveBedrms*, *AveOccup*, *Latitude*, and *Longitude*. This result suggests that simple main effects are not sufficient to explain the housing target well, and location-density interactions play an important role.

For the tree-based model, I used XGBoost [3] with a randomized search over boosting hyperparameters. The best final XGBoost configuration used 350 trees, maximum depth 2, learning rate 0.15, subsample 1.0, column sample ratio 0.7, minimum child weight 3, $\lambda = 1.0$, and $\alpha = 0.0$. The Huber baseline was intentionally kept simpler, using degree-1 features on the raw target, because its main role in this assignment was to test whether a robust loss could mitigate outlier sensitivity and residual pathologies.

2.3 Evaluation

I evaluated all models using RMSE, MAE, R^2 , adjusted R^2 , MAPE, and explained variance. Model comparison used 5-fold cross-validation repeated 3 times, producing 15 paired fold-level measurements per model. Table 1 reports the mean and standard deviation across these repeated folds.

Table 1: Repeated cross-validation results for Question 1.

Model	RMSE	MAE	R^2	Adj. R^2	MAPE	Exp. Var.
XGBoost Regressor	0.5166 ± 0.0144	0.3452 ± 0.0073	0.8009 ± 0.0093	0.8004 ± 0.0093	0.1850 ± 0.0049	0.8022 ± 0.0091
Linear Regression	0.6528 ± 0.0161	0.4500 ± 0.0077	0.6819 ± 0.0149	0.6776 ± 0.0151	0.2459 ± 0.0053	0.6846 ± 0.0145
RidgeCV	0.6957 ± 0.0235	0.4845 ± 0.0196	0.6382 ± 0.0302	0.6333 ± 0.0306	0.2698 ± 0.0126	0.6426 ± 0.0295
Elastic Net CV	0.7146 ± 0.0296	0.5011 ± 0.0202	0.6187 ± 0.0290	0.6135 ± 0.0294	0.2800 ± 0.0115	0.6237 ± 0.0274
LassoCV	0.7202 ± 0.0317	0.5054 ± 0.0209	0.6126 ± 0.0320	0.6074 ± 0.0325	0.2827 ± 0.0120	0.6180 ± 0.0302
Huber Regressor	1.4543 ± 1.1438	0.5651 ± 0.0231	-1.4667 ± 3.7966	-1.4727 ± 3.8058	0.3222 ± 0.0182	-1.4558 ± 3.7918

XGBoost was the clear winner on every reported metric. Its advantage was not only numerical but also statistically consistent across folds. Paired tests on fold-level RMSE showed

that XGBoost significantly outperformed Linear Regression ($p = 4.79 \times 10^{-18}$), RidgeCV ($p = 2.24 \times 10^{-12}$), LassoCV ($p = 2.57 \times 10^{-14}$), Elastic Net ($p = 2.38 \times 10^{-14}$), and Huber Regressor ($p = 6.62 \times 10^{-3}$). The corresponding Wilcoxon signed-rank tests also favored XGBoost, with $p = 6.1 \times 10^{-5}$ in all pairwise comparisons reported by the pipeline.

After model selection, I evaluated the best cross-validated model and the robust Huber baseline once on the holdout test set. Table 2 confirms that the cross-validation ranking transferred to the final split: XGBoost achieved the best test performance, with $\text{RMSE} = 0.5052$ and $R^2 = 0.8027$, while Huber remained substantially worse.

Table 2: Single holdout test-set results for the selected best model and the robust baseline.

Model	RMSE	MAE	R^2	Adj. R^2	MAPE	Exp. Var.
XGBoost Regressor	0.5052	0.3378	0.8027	0.8023	0.1765	0.8045
Huber Regressor	0.8861	0.5404	0.3929	0.3917	0.2956	0.4031

Residual diagnostics were also computed on the final holdout predictions. For XGBoost, the Breusch-Pagan test returned a p-value of 1.67×10^{-22} , and the Jarque-Bera test returned $p \approx 0$, indicating remaining heteroscedasticity and non-normal residuals. Huber did not solve these issues either; its Breusch-Pagan p-value was 3.38×10^{-132} , and its Jarque-Bera p-value was again effectively zero. Figure 2 visualizes the residual patterns and Q-Q behavior for the selected model.

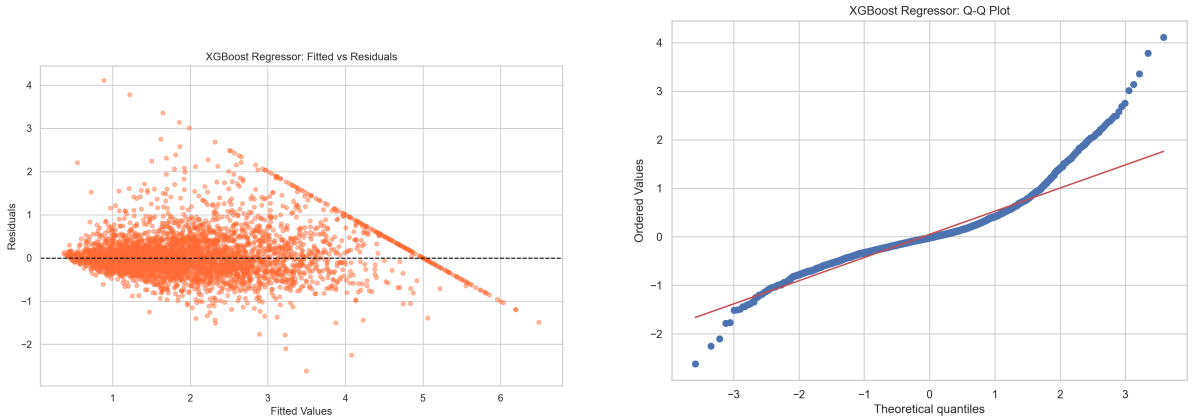


Figure 2: Residual diagnostics for the final XGBoost model. The residual plot shows non-constant spread, and the Q-Q plot shows clear departures from Gaussian residuals.

2.4 Discussion

The results show a clear bias-variance trade-off. Linear Regression had higher bias than the boosted tree model but still generalized reasonably well and remained much more stable than the regularized degree-2 variants. Ridge, Lasso, and Elastic Net all underperformed the unregularized linear baseline here, suggesting that the chosen polynomial expansion plus shrinkage introduced too much bias relative to the structure present in the data. In contrast, XGBoost captured useful non-linearities without the same feature-space explosion and therefore achieved the best predictive accuracy and the tightest fold-to-fold variance.

The robust regression experiment is informative precisely because it did *not* fix the core problem. Although Huber is designed to reduce the influence of extreme observations, it still performed far worse than XGBoost and did not eliminate heteroscedasticity or non-normality in the residuals. This suggests that the residual violations are not caused only by a handful

of extreme outliers; they are more likely tied to real non-linear structure, omitted interactions, and regional heterogeneity in the housing market.

Overall, the best-performing model in this experiment was the tree-based boosted regressor. It provided the strongest predictive performance, statistically significant improvements over all baselines, and the best final test-set results. The main limitation is interpretability: while the linear models are easier to explain coefficient-by-coefficient, they did not match the predictive quality of boosted trees. If I were extending this question further, I would keep XGBoost as the primary predictive model and pair it with model-agnostic interpretation tools such as SHAP to recover local and global explanations without sacrificing performance.

3 Question 2: Classification Under Extreme Imbalance

3.1 Problem Setup

For Question 2, I used the credit-card fraud dataset stored at `data/external/creditcard.csv` [12], following the unbalanced-classification setting described by Dal Pozzolo et al. [4]. The task is binary fraud detection with target column `Class`, where the positive class corresponds to fraudulent transactions. The dataset is extremely imbalanced, with a positive rate of 0.030569 and an imbalance ratio of approximately 31.71:1 in favor of the negative class. I kept a single stratified 20% holdout split for final evaluation and restricted all model selection, resampling, calibration, and threshold tuning to the training portion only.

Leakage prevention was handled explicitly. The final test set was never touched during model comparison. For each cross-validation fold, resampling was performed only inside an `imblearn` pipeline after preprocessing, so synthetic or undersampled observations were generated from training-fold data only. I scaled `Time` and `Amount` while leaving the PCA-style `V1-V28` features as passthrough numeric inputs.

3.2 Models and Sampling Strategies

I compared four classifier families: Logistic Regression, Random Forest, XGBoost [3], and a feed-forward MLP classifier. Each model was evaluated under five imbalance-handling strategies: no resampling, SMOTE [2], ADASYN [7], random undersampling, and cost-sensitive learning. Cost-sensitive learning used `class_weight="balanced"` for Logistic Regression and Random Forest, `scale_pos_weight` for XGBoost, and balanced sample weights for the MLP.

Model ranking used 5-fold stratified cross-validation on the training split. Candidates were ordered primarily by PR-AUC, then MCC, then recall, since PR-AUC is more informative than ROC-AUC under severe imbalance and the downstream objective is recall-first fraud detection. The top two uncalibrated candidates were XGBoost Classifier + No Resampling and XGBoost Classifier + Cost Sensitive.

Table 3: Top Question 2 cross-validation results.

Candidate	PR-AUC	MCC	Recall	Precision	Balanced Acc.
XGBoost Classifier + No Resampling	0.9137 $\pm$ 0.0245	0.9088	0.8467	0.9818	0.9231
XGBoost Classifier + Cost Sensitive	0.9104 $\pm$ 0.0230	0.9040	0.8625	0.9540	0.9306

3.3 Evaluation and Calibration

All candidate models were evaluated using precision, recall, macro F1, micro F1, ROC-AUC, PR-AUC, MCC, and balanced accuracy. After cross-validation, I recalibrated the best two candidates using both Platt scaling (sigmoid) and isotonic regression on a calibration-only

validation slice drawn from the training partition. The better calibration method for each candidate was selected by Brier score.

The best calibrated variant overall was XGBoost Classifier + Cost Sensitive with isotonic calibration, achieving a calibration-set Brier score of 0.003168. Threshold selection was then performed on the calibration split using two rules: the F1-max threshold and a recall-first cost-sensitive threshold with false-negative cost 5 and false-positive cost 1. Because missing fraud is more costly than over-flagging a legitimate transaction, I used the cost-sensitive threshold as the default operating point.

Table 4: Threshold comparison for the selected final model on the calibration split used for threshold selection.

Policy	Threshold	Precision	Recall	False Negatives	False Positives
Recall-first cost-sensitive	0.5000	0.9811	0.9123	5	1
F1-max	0.5000	0.9811	0.9123	5	1

Table 4 is not the final test result; it reports the calibration split used to choose the operating threshold without touching the holdout set. The final selected model was then evaluated once on the untouched holdout set. Under the default recall-first threshold, it achieved precision 0.9268, recall 0.8000, PR-AUC 0.8558, MCC 0.8571, balanced accuracy 0.8990, and Brier score 0.007574. Figures 3 and 4 visualize the final holdout decision behavior.

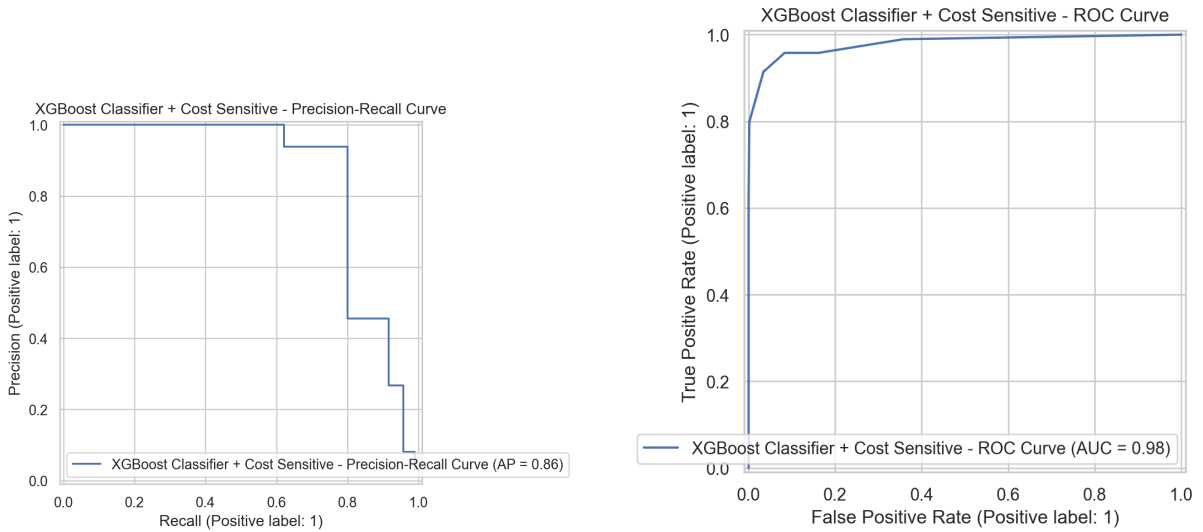


Figure 3: Precision-recall and ROC curves for the final calibrated Question 2 model on the holdout set.

3.4 Discussion

The results show why PR-AUC and recall-aware thresholding matter for extreme imbalance. A model can look acceptable under ROC-AUC while still failing to identify enough fraud cases at a usable operating threshold. The final pipeline therefore emphasized PR-AUC during model ranking and then used a cost-sensitive threshold to explicitly favor recall. This choice reflects the application setting: missing a fraudulent transaction is typically more costly than reviewing an extra flagged transaction.

Calibration was also necessary because raw classifier scores are not always well aligned with true fraud probabilities. By comparing sigmoid and isotonic calibration on a dedicated

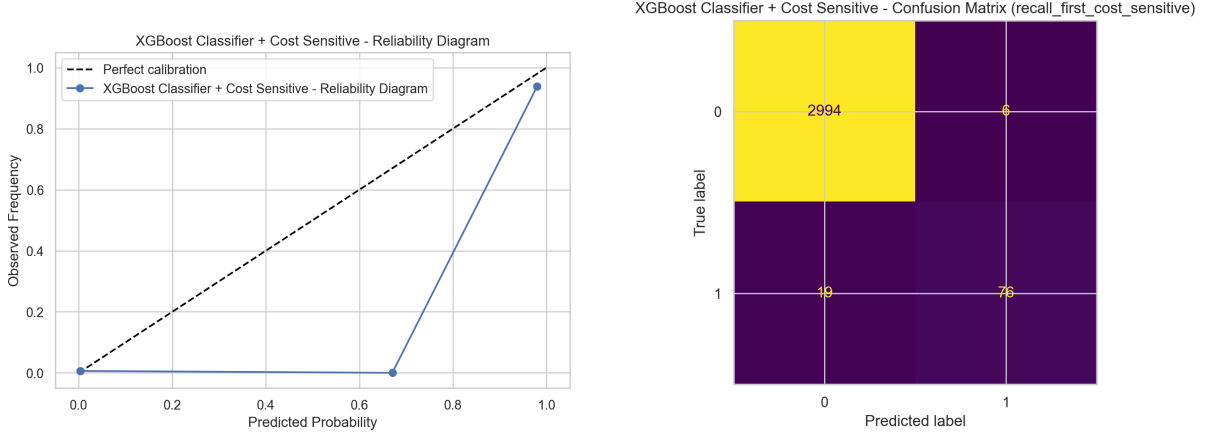


Figure 4: Reliability diagram and confusion matrix for the final calibrated Question 2 model at the recall-first operating threshold.

validation slice, the final workflow separates ranking quality from probability quality. This improves both the reliability diagram and the downstream threshold decision. In practice, the most meaningful trade-off is between recall and operational alert volume, so the final report should interpret false positives as review workload and false negatives as direct fraud risk.

4 Question 3: Dimensionality Reduction

4.1 Problem Setup

This question compares linear, kernel-based, manifold-learning, and neural dimensionality reduction methods on Fashion-MNIST [21]. The experiment uses a balanced subset of 5,000 training examples and 1,000 test examples, sampled equally from the ten classes so that visual separation and downstream k-NN accuracy are not dominated by class-frequency effects. Images are flattened from 28×28 pixels into 784-dimensional vectors and scaled to $[0, 1]$.

4.2 Methods

The implemented methods are PCA, RBF Kernel PCA, t-SNE [20], UMAP [13], and a fully connected undercomplete autoencoder [9]. PCA and Kernel PCA reduce directly to two components. t-SNE is evaluated with perplexities 5, 30, and 50. UMAP is evaluated over $n_neighbors \in \{10, 15, 30\}$ and $min_dist \in \{0.0, 0.1, 0.5\}$. The autoencoder uses encoder layers of 256, 128, and 64 units, a two-dimensional bottleneck, a symmetric decoder, and mean-squared reconstruction loss for 20 epochs.

4.3 Evaluation

Table 5 summarises the reconstruction and downstream classification results from the run 20260501-165404_fashion_mnist_baseline. The autoencoder obtains the lowest reconstruction error. For downstream classification, however, the best result is obtained by t-SNE with perplexity 30, not by the t-SNE configuration selected by manifold-quality metrics.

For manifold quality, the selected t-SNE configuration is perplexity 50, with trustworthiness 0.9804, continuity 0.9765, and Kruskal stress 0.3211. This should not be interpreted as the best t-SNE setting for every metric: perplexity 30 gives the highest downstream k-NN accuracy, 0.7858 ± 0.0135 . The selected UMAP configuration is $n = 15$, $min_dist=0.1$, with trustworthiness 0.9776, continuity 0.9742, Kruskal stress 0.4031, and k-NN accuracy 0.7204 ± 0.0101 .

Table 5: Question 3 reconstruction MSE and 5-fold k-NN accuracy on 2D embeddings.

Method	Reconstruction MSE	k-NN accuracy
PCA	0.0462	0.4902 ± 0.0089
Kernel PCA	0.0660	0.4944 ± 0.0057
t-SNE, perplexity 30	—	0.7858 ± 0.0135
t-SNE, selected perplexity 50	—	0.7748 ± 0.0111
UMAP, $n = 15$, $\text{min_dist}=0.1$	—	0.7204 ± 0.0101
Autoencoder	0.0339	0.5884 ± 0.0116

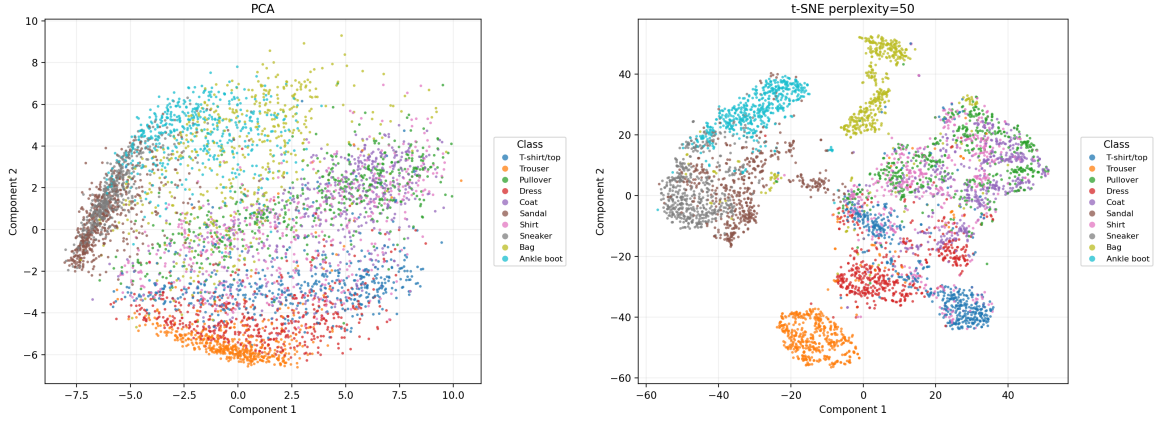


Figure 5: PCA and selected t-SNE 2D embeddings coloured by Fashion-MNIST class.

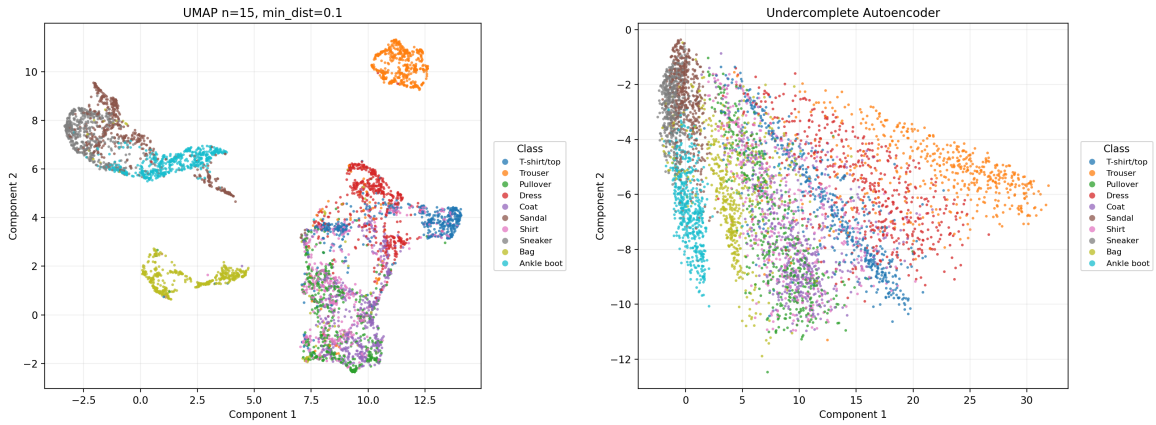


Figure 6: Selected UMAP embedding and learned autoencoder latent space.

4.4 Discussion

On this balanced subset, PCA is the fastest method, but its linear projection cannot separate visually similar clothing categories. Kernel PCA improves the representation only slightly for k-NN and requires a kernel matrix, so its memory cost grows poorly with sample count. t-SNE gives the best downstream classification accuracy at perplexity 30, while perplexity 50 is preferred only under the manifold-quality selection rule. Its main limitation is that it is non-parametric and does not naturally transform new samples.

UMAP is competitive on neighbourhood preservation and gives a reusable fitted transformer, but its selected setting is below t-SNE on k-NN accuracy. The autoencoder gives the best reconstruction MSE and a reusable encoder, but its 2D bottleneck is less class-separating than t-SNE and UMAP. The learned latent space is still useful because it captures continuous visual factors such as footwear versus clothing and broad silhouette differences. Autoencoders would be preferable when the system must embed many future samples after training, when the representation should be integrated into a larger neural pipeline, or when deeper architectures can learn non-linear semantic factors that PCA and t-SNE cannot express directly.

5 Question 4: Clustering

5.1 Problem Setup

The configured Wholesale Customers CSV was not available in the repository, so this experiment uses the assignment’s optional labelled `optdigits` setting through the local sklearn digits loader [1]. The dataset contains 1,797 handwritten digit images represented by 64 pixel-intensity features. Features are standardised before clustering. The digit labels are not used to fit the clustering models; they are used only for optional external validation with ARI, NMI, and Fowlkes-Mallows scores.

5.2 Methods

Four clustering algorithms are implemented. K-Means is evaluated over $k = 2, \dots, 10$ using inertia, silhouette [18], and gap-statistic diagnostics; the final choice is $k = 10$. Gaussian Mixture Models are selected with BIC/AIC, resulting in 5 components. DBSCAN [5] uses a k-distance graph with `min_samples=5` and selects $\epsilon = 5.772$ from the quantile grid. Agglomerative clustering uses Ward linkage and selects $k = 10$ by silhouette, with a truncated dendrogram saved for inspection.

The ensemble method builds a co-association matrix from K-Means, a nearby K-Means variant, GMM, and DBSCAN, then reclusters the matrix with average-linkage agglomerative clustering. This gives a direct comparison between individual methods and a simple consensus clustering.

5.3 Evaluation

Table 6 shows that internal and external criteria disagree. DBSCAN has the highest silhouette and stability, but it finds only two non-noise clusters with about 5.1% noise, so it does not recover the ten digit classes. K-Means has the best CH score and lowest DB score, while Agglomerative clustering has the strongest external validation, with ARI [10] 0.6643, NMI 0.7956, and FMI 0.7111. The ensemble improves over GMM and DBSCAN in ARI/NMI, but it does not outperform the best individual method.

Table 6: Question 4 clustering metrics on the optdigits dataset. CH is Calinski-Harabasz, DB is Davies-Bouldin, and FMI is Fowlkes-Mallows.

Method	Silhouette	CH	DB	ARI	NMI	FMI	Stability ARI
K-Means	0.1469	115.04	1.89	0.4684	0.6279	0.5352	0.8064
GMM	0.0923	70.43	2.68	0.0359	0.1932	0.2682	0.4639
DBSCAN	0.2522	23.88	3.54	0.0009	0.0189	0.3000	0.8579
Agglomerative	0.1253	105.83	1.97	0.6643	0.7956	0.7111	0.7716
Ensemble	0.0787	78.48	1.95	0.2692	0.5039	0.4142	0.4880

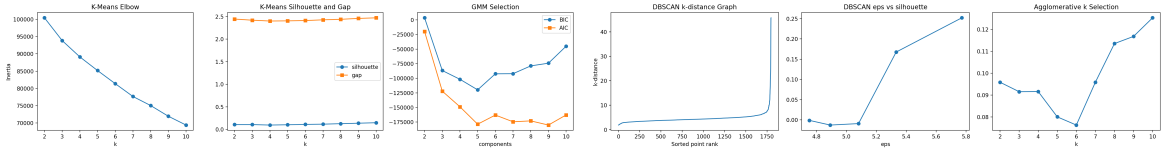


Figure 7: Q4 model-selection diagnostics for K-Means/GMM and DBSCAN/Agglomerative clustering.

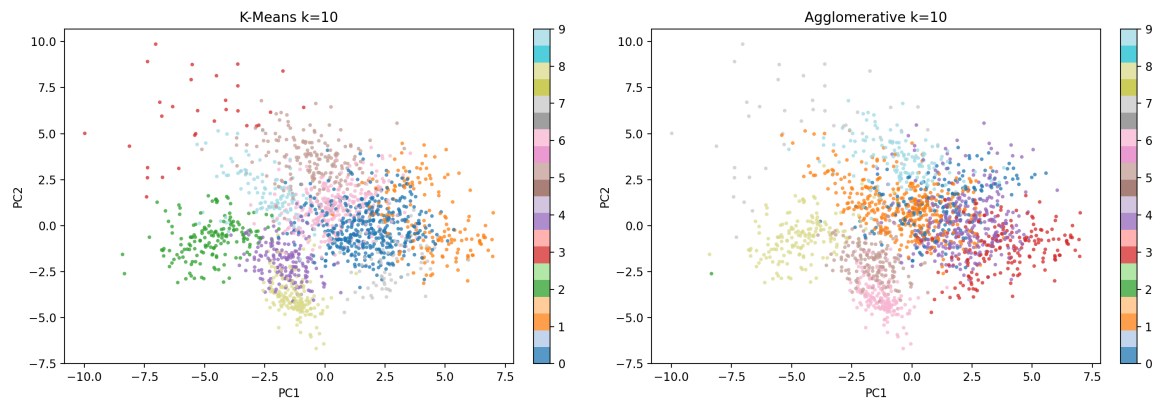


Figure 8: PCA projections coloured by K-Means and Agglomerative cluster assignments.

5.4 Discussion

The results illustrate why clustering should not be judged by one metric alone. K-Means assumes roughly spherical clusters and performs reasonably when $k = 10$, but digit shapes overlap in the pixel space, limiting ARI. GMM assumes Gaussian mixture components; BIC favours only 5 components, which under-segments the ten digit classes. DBSCAN is stable and obtains the highest silhouette because it separates a dense core from outliers, but this density-based structure is not aligned with the semantic digit labels. Ward agglomerative clustering best matches the external labels, suggesting that hierarchical merges capture digit similarity more effectively than the flat Gaussian or density assumptions in this feature space. The ensemble is useful as a robustness check, but its consensus is pulled down by weak GMM and DBSCAN partitions.

6 Question 5: Neural Networks

6.1 Problem Setup

This experiment compares a deep MLP, a CNN trained from scratch, and a transfer-learning ResNet18 baseline on CIFAR-10 [11]. To keep the run feasible on the available device, the pipeline uses a balanced subset rather than the full dataset: 2,500 training images, 500 validation images, and 500 test images. Images are normalised with CIFAR-10 statistics for the MLP/CNN and ImageNet statistics for ResNet18. The reported run uses timestamp 20260502-160350. The subset was sampled equally from the ten CIFAR-10 classes. I used this reduced but class-balanced protocol to keep repeated model training, Optuna search, FGSM evaluation, and interpretability experiments reproducible within the assignment runtime. Therefore, the results are intended for controlled comparison between architectures rather than as full CIFAR-10 benchmark scores.

6.2 Models and Regularisation

The MLP flattens each 32×32 RGB image and uses four hidden layers of 512 units, batch normalisation, ReLU activations, dropout, Adam, weight decay, and early stopping. The CNN is trained from scratch with four convolutional layers, batch normalisation, max pooling, dropout, and data augmentation through random crop, horizontal flip, and small rotations. The ResNet18 baseline [8] starts from ImageNet weights and fine-tunes only the final residual block and classifier head. An Optuna-based compact search was used for learning rate, batch size, dropout, and weight decay; all three final models selected learning rate 0.001, batch size 64, and weight decay 0.0001. The MLP and ResNet18 selected dropout 0.3, while the CNN selected dropout 0.5.

Each final model was configured for 4 epochs and actually trained for all 4 epochs; early stopping did not trigger. The runtime values below are final-model training times only. Search runtimes are saved separately in the generated artefacts.

6.3 Evaluation

Table 7: Question 5 test metrics on the class-balanced CIFAR-10 subset used for controlled neural-network comparison.

Model	Accuracy	Macro-F1	Top-5 error	Train runtime (s)
Deep MLP	0.348	0.3444	0.144	1.10
CNN from scratch	0.408	0.4019	0.104	2.64
ResNet18 transfer	0.798	0.7977	0.008	5.90

Table 7 shows a clear benefit from transfer learning on this subset. The scratch CNN improves over the MLP by using local spatial structure, while ResNet18 has the strongest accuracy, macro-F1, and top-5 performance. These results should not be read as full CIFAR-10 benchmark scores because both the training and test sets are intentionally reduced.

Evaluation and FGSM [6] use `model.eval()`. For FGSM, each normalised batch is converted back to $[0, 1]$ pixel space, perturbed by $\epsilon = 0.03$, clamped to valid pixels, and then re-normalised before the adversarial forward pass. ResNet18 is strongest on most clean classes, especially automobile, frog, ship, and truck. Cat remains the hardest class even for the transfer model, with recall 0.58. Under FGSM, all three models degrade sharply, so clean accuracy should not be treated as evidence of robustness.

Table 8: Per-class recall and FGSM robustness. FGSM uses $\epsilon = 0.03$ in pixel space.

Class / Metric	MLP	CNN	ResNet18
airplane	0.44	0.60	0.84
automobile	0.26	0.48	0.90
bird	0.28	0.26	0.72
cat	0.18	0.16	0.58
deer	0.22	0.18	0.70
dog	0.26	0.50	0.74
frog	0.42	0.38	0.94
horse	0.44	0.58	0.78
ship	0.60	0.56	0.92
truck	0.38	0.38	0.86
FGSM accuracy	0.116	0.110	0.114

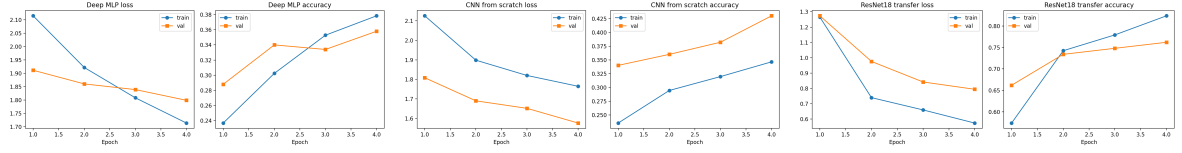


Figure 9: Training and validation loss/accuracy curves for the three neural models.

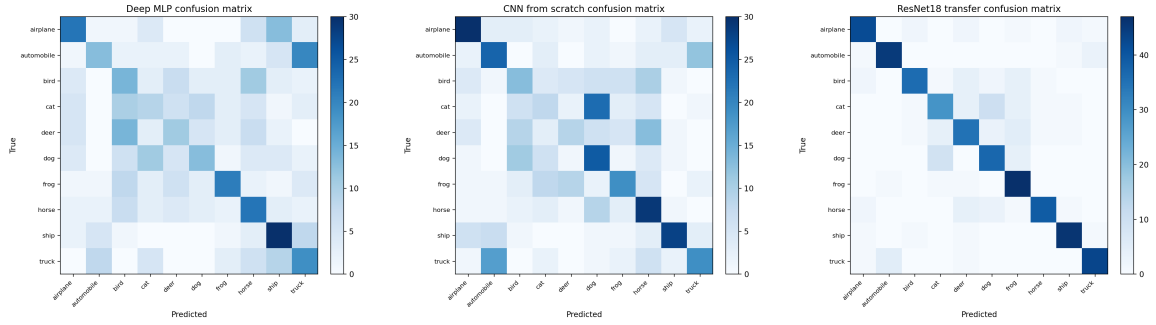


Figure 10: Confusion matrices for MLP, scratch CNN, and transfer ResNet18.

6.4 Interpretability and Discussion

The pipeline saves ten misclassified examples for each model. Grad-CAM [19] is applied to selected CNN and ResNet18 mistakes, and LIME [17] is applied to one MLP mistake with a limited perturbation budget. The LIME figure should be interpreted only as a local indication of positive superpixel regions for that prediction, not as a global explanation of the MLP.

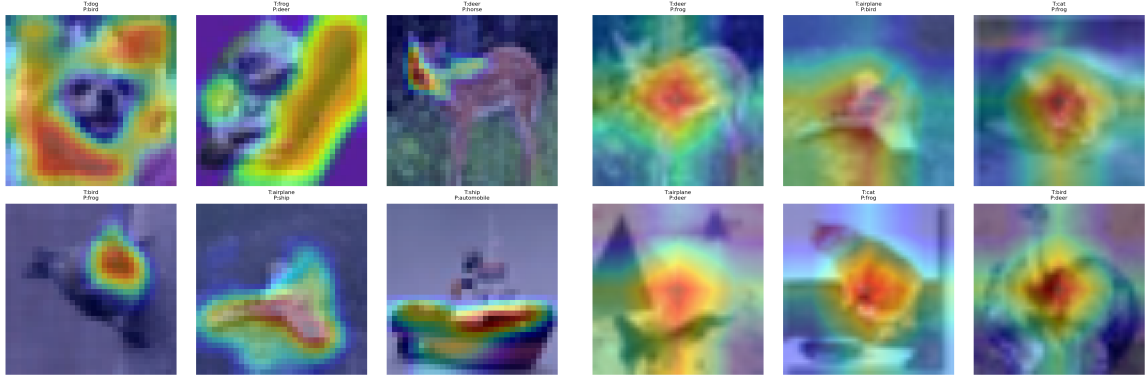


Figure 11: Grad-CAM heatmaps for selected misclassified CNN and ResNet18 examples.

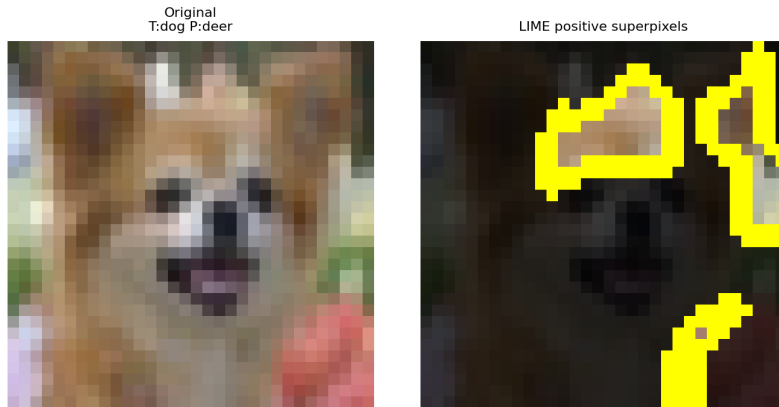


Figure 12: Original MLP mistake and LIME-selected positive superpixels for that prediction.

Overall, the results follow the expected model-capacity ordering on the reduced dataset: the MLP is fastest but ignores image locality, the scratch CNN benefits from convolution and augmentation, and the pretrained ResNet18 gives the best clean classification performance. The robustness result is more cautious: all models are vulnerable to the same pixel-space FGSM attack, and the experiment is too small to support broader robustness claims.

7 Conclusion

Across the five questions, the strongest results came from methods that matched the structure of the data rather than from the most generic baseline. In Question 1, XGBoost gave the best California Housing regression performance, showing that non-linear interactions mattered more than simple polynomial linear models. In Question 2, the calibrated cost-sensitive XGBoost model was the most appropriate fraud detector because the evaluation emphasized PR-AUC, recall, and threshold behaviour under extreme imbalance.

For representation learning, Question 3 showed that dimensionality-reduction goals can disagree. The autoencoder reconstructed Fashion-MNIST best, while t-SNE gave the strongest two-dimensional downstream k-NN accuracy and UMAP remained a practical reusable manifold method. Question 4 similarly showed that clustering metrics must be interpreted carefully: DBSCAN looked best by silhouette, but agglomerative clustering matched the digit labels best under external validation.

Question 5 extended this pattern to neural networks. On a balanced CIFAR-10 subset, transfer learning with ResNet18 clearly outperformed the scratch CNN and deep MLP on clean accuracy, but FGSM testing showed that clean performance did not imply adversarial robustness. Overall, the main lesson is that model choice, validation metric, and evaluation protocol have to be aligned with the task: the best method changed across regression, imbalance-aware classification, dimensionality reduction, clustering, and image classification.

References

- [1] Ethem Alpaydin and Cenk Kaynak. Optical recognition of handwritten digits data set. UCI Machine Learning Repository, 1998. Dataset distributed in scikit-learn as the digits loader.
- [2] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.
- [3] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.
- [4] Andrea Dal Pozzolo, Olivier Caelen, Reid A. Johnson, and Gianluca Bontempi. Calibrating probability with undersampling for unbalanced classification. In *IEEE Symposium Series on Computational Intelligence*, pages 159–166, 2015.
- [5] Martin Ester, Hans-Peter Kriegel, Jorg Sander, and Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, pages 226–231, 1996.
- [6] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *International Conference on Learning Representations*, 2015.
- [7] Haibo He, Yang Bai, Edwardo A. Garcia, and Shutao Li. ADASYN: Adaptive synthetic sampling approach for imbalanced learning. In *IEEE International Joint Conference on Neural Networks*, pages 1322–1328, 2008.
- [8] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016.

- [9] Geoffrey E. Hinton and Ruslan R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006.
- [10] Lawrence Hubert and Phipps Arabie. Comparing partitions. *Journal of Classification*, 2:193–218, 1985.
- [11] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.
- [12] Machine Learning Group, Université Libre de Bruxelles. Credit card fraud detection dataset. Kaggle dataset, 2018. European cardholder transactions used for fraud-detection benchmarking.
- [13] Leland McInnes, John Healy, and James Melville. UMAP: Uniform manifold approximation and projection for dimension reduction. *arXiv preprint arXiv:1802.03426*, 2018.
- [14] R. Kelley Pace and Ronald Barry. Sparse spatial autoregressions. *Statistics & Probability Letters*, 33(3):291–297, 1997.
- [15] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.
- [16] Fabian Pedregosa, Gael Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Edouard Duchesnay. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.
- [17] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. Why should i trust you? explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144, 2016.
- [18] Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987.
- [19] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision*, pages 618–626, 2017.
- [20] Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of Machine Learning Research*, 9:2579–2605, 2008.
- [21] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: A novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.